

INTRODUCTION:

This is a 2D Unity-based arcade game where the player controls a character and tries to avoid obstacles while continuously moving forward. The player can control the character using keyboard, mouse, or touch input to move upward, while gravity constantly pulls the character downward.

The game includes a scoring system, progressive difficulty, obstacle spawning, pause and continue functionality, game-over handling, sound effects, and animated player sprites. Obstacles are spawned at random heights and move from right to left. As the player's score increases, the difficulty level increases and the obstacle speed becomes faster.

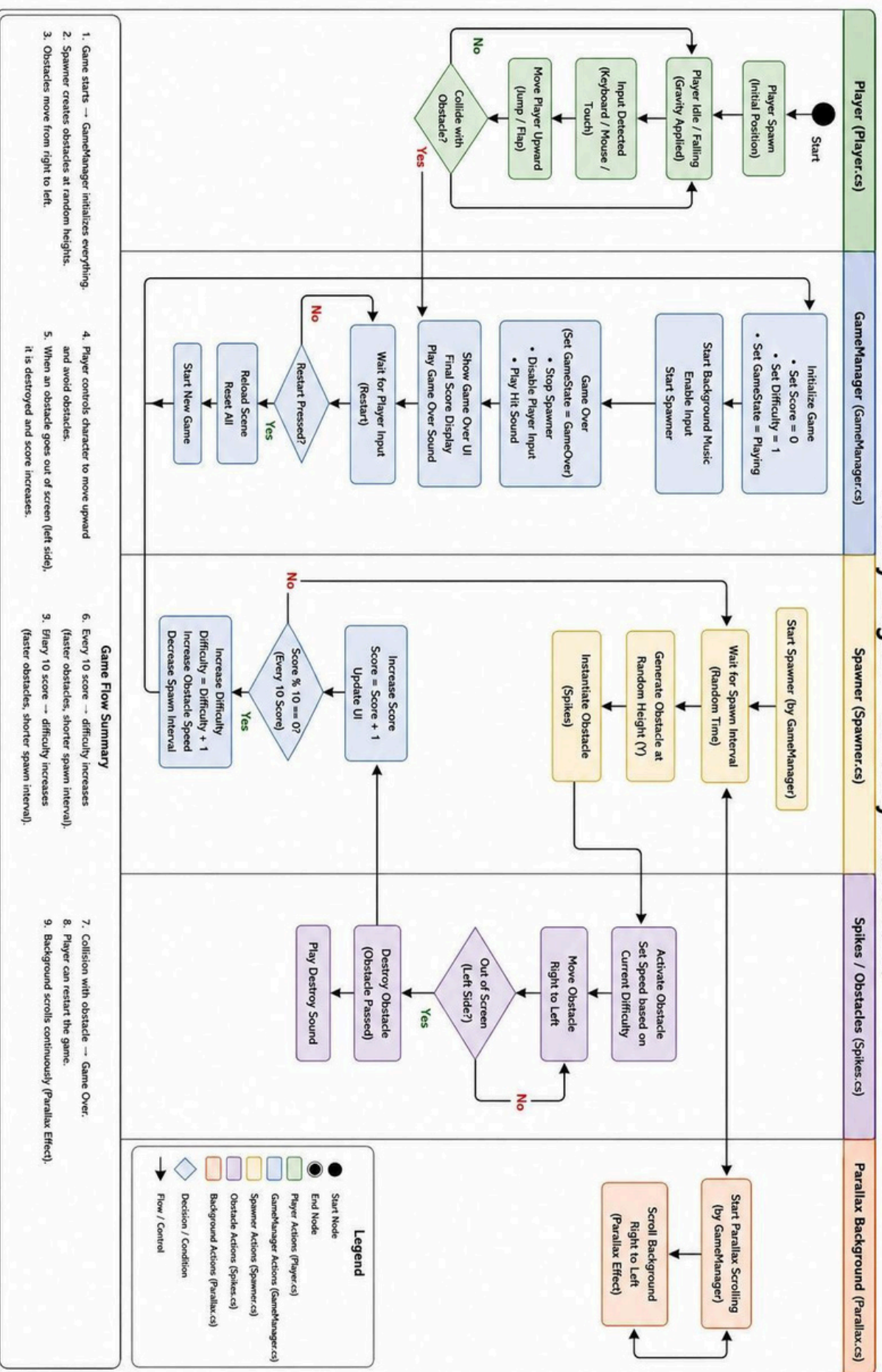
The project is implemented using multiple C# scripts, including GameManager, Player, Spawner, Spikes, and Parallax. The GameManager controls the overall game state, score, difficulty level, pause/continue system, game-over state, and sound effects. The Player class handles player movement, input, sprite animation, and collision detection. The Spawner class creates obstacles at regular intervals with random vertical positions, while the Spikes class controls obstacle movement and destruction. The Parallax class creates a scrolling background effect.

Features list:

User 1: Player

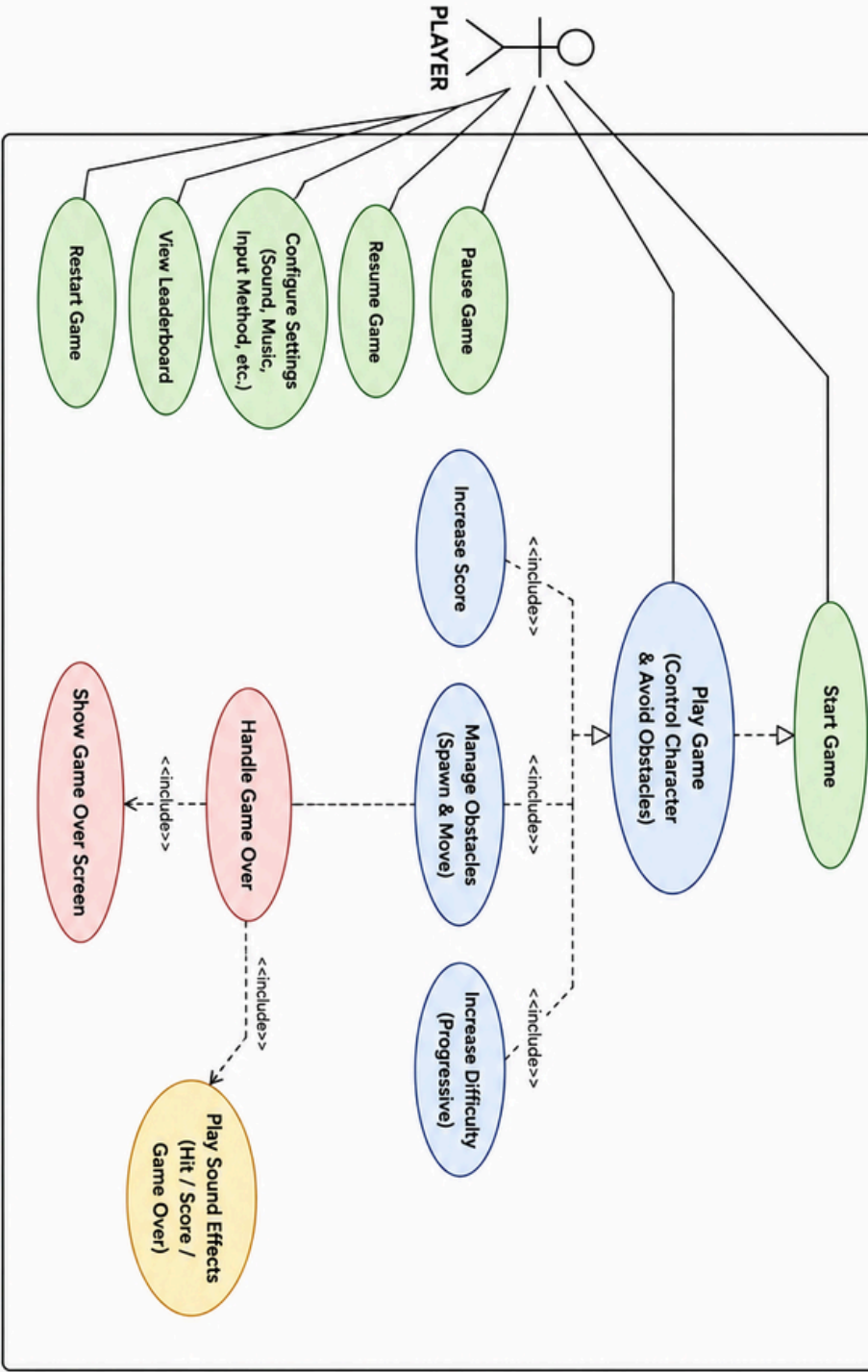
User 2: Developer

Activity Diagram – 2D Unity Arcade Game



2D ARCADE GAME – USE CASE DIAGRAM

2D ARCADE GAME SYSTEM



MAIN FLOW

1. Player starts the game.
2. Player controls the character to move up and avoid obstacles.
3. System spawns obstacles and moves them from right to left.
4. Player scores points for survival.
5. System increases difficulty as score increases.
6. If player hits an obstacle → Handle Game Over.
7. Game Over Screen is shown and sound is played.

ACTOR

PLAYER : The person who plays the game.

LEGEND

- Player Initiated Use Case
- Gameplay Use Case
- Game State Use Case
- System Service Use Case
- Association (Actor participates in Use Case)
- - - - - <<include>> (Mandatory behavior)
- - - - - <<extend>> (Optional behavior)
- - - - - ▾ Generalization

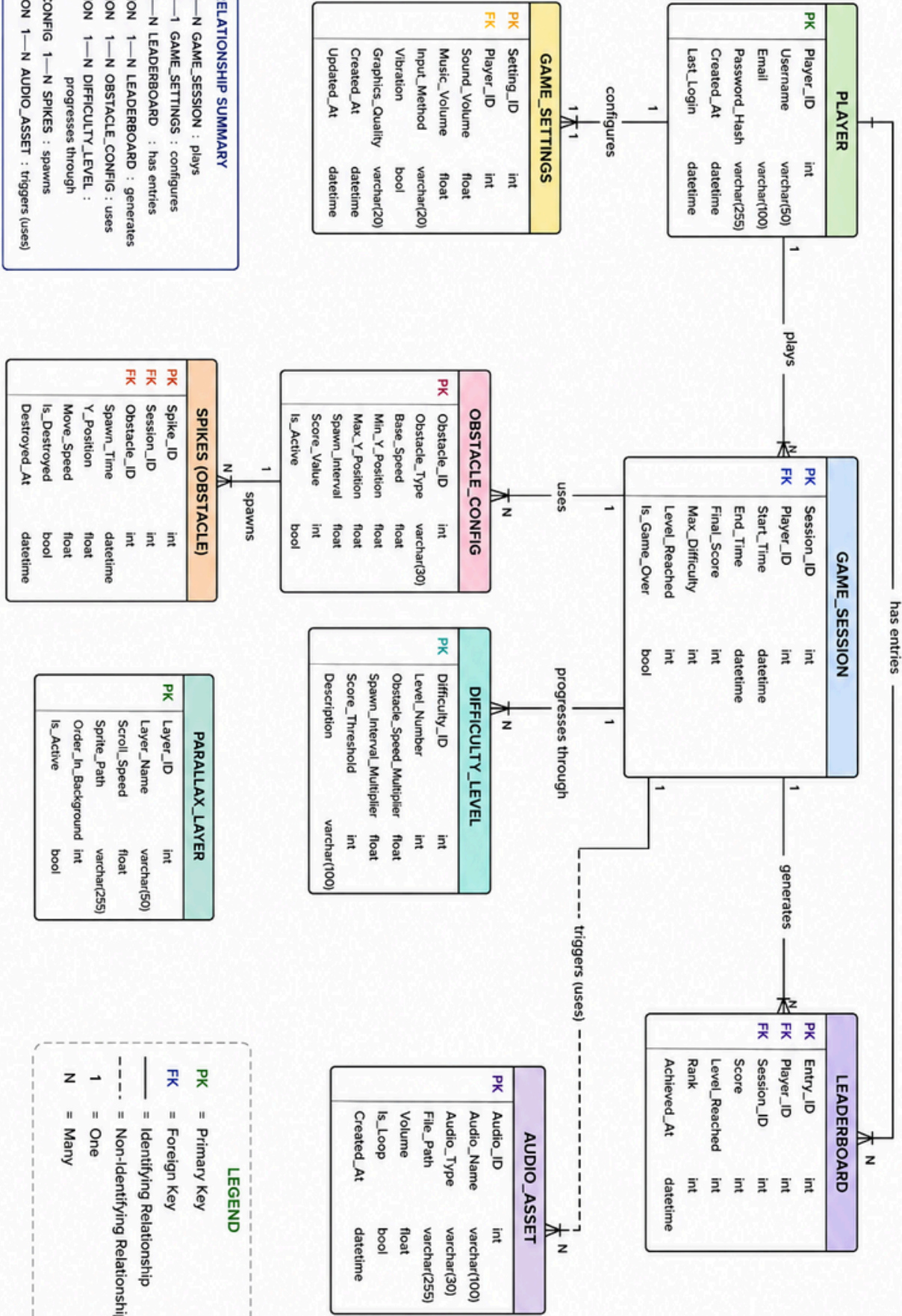
RELATIONSHIP SUMMARY

- Play Game <<include>> Increase Score
- Play Game <<include>> Manage Obstacles
- Play Game <<include>> Increase Difficulty
- Handle Game Over <<include>> Show Game Over Screen
- Handle Game Over <<include>> Play Sound Effects

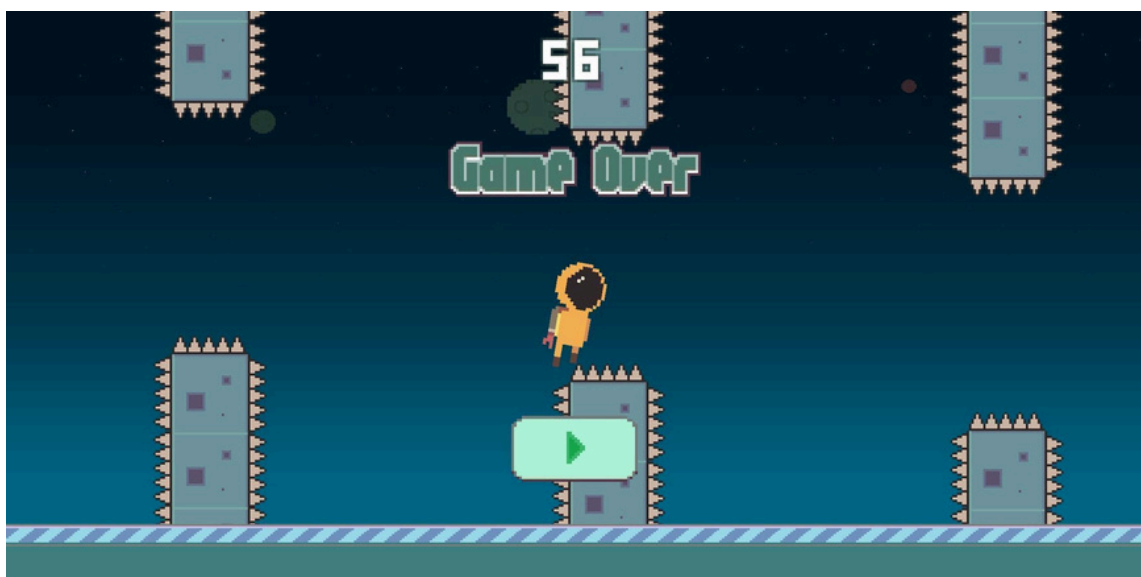
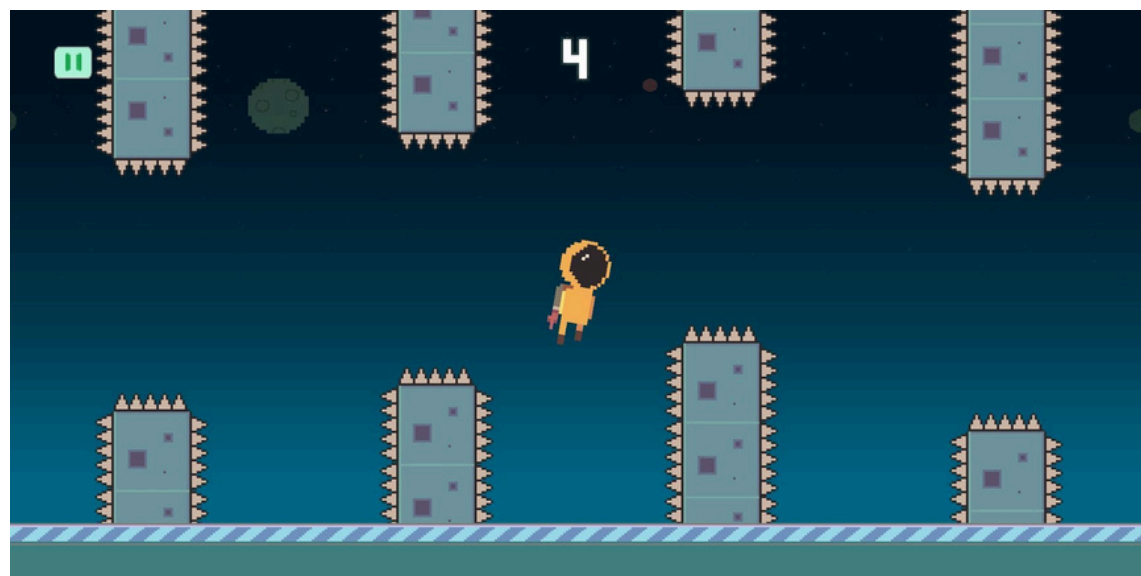
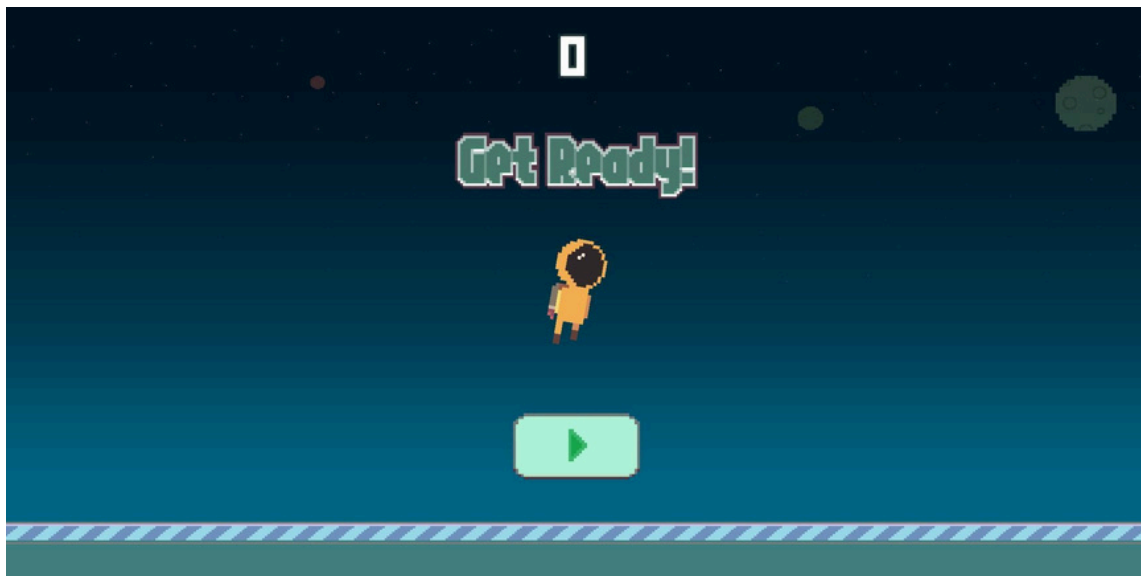
DESCRIPTION

The player starts the game and controls the character to move upward and avoid obstacles. The system manages obstacle spawning and movement, increases difficulty as the score increases, and handles game over, sounds, pause/resume, settings, restart, and leaderboard.

2D ARCADE GAME – ER DIAGRAM



Screenshots:



2D ARCADE GAME — DATABASE NORMALIZATION

Goal: Convert the ER-diagram relations into well-structured relations that satisfy 1NF → 2NF → 3NF, while keeping each fact stored in one appropriate place.

1NF — First Normal Form

All attributes are atomic (single values), there are no repeating groups, and every table has a primary key. The given entities therefore satisfy 1NF as designed: PLAYER, GAME_SESSION, GAME_SETTINGS, LEADERBOARD, OBSTACLE_CONFIG, SPIKES, DIFFICULTY_LEVEL, AUDIO_ASSET and PARALLAX_LAYER.

2NF — Second Normal Form

2NF requires 1NF plus no non-key attribute depending on only part of a composite key. The design uses single-column primary keys (Player_ID, Session_ID, Setting_ID, Entry_ID, etc.), so there is **no partial dependency**. Therefore the relations remain in their current entity-based form.

3NF — Third Normal Form

3NF requires 2NF plus no non-key attribute depending on another non-key attribute. Attributes are kept with the entity they describe—for example, password information stays in PLAYER, sound properties stay in AUDIO_ASSET, and difficulty properties stay in DIFFICULTY_LEVEL. Thus the normalized relations below are in 3NF, assuming the stated business rules.

Normalized Relations (3NF)

Relation	Primary Key	Foreign Keys	Non-key attributes
PLAYER	Player_ID	—	Username, Email, Password_Hash, Created_At, Last_Login
GAME_SETTINGS	Setting_ID	Player_ID → PLAYER	Sound_Volume, Music_Volume, Input_Method, Vibration, Graphics_Quality, Created_At, Updated_At
GAME_SESSION	Session_ID	Player_ID → PLAYER; Difficulty_ID → DIFFICULTY_LEVEL*	Start_Time, End_Time, Final_Score, Max_Difficulty, Level_Reached, Is_Game_Over
LEADERBOARD	Entry_ID	Player_ID → PLAYER; Session_ID → GAME_SESSION	Score, Level_Reached, Rank, Achieved_At
DIFFICULTY_LEVEL	Difficulty_ID	—	Level_Number, Obstacle_Speed_Multiplier, Spawn_Interval, Multiplier, Score_Threshold, Description
OBSTACLE_CONFIG	Obstacle_ID	Session_ID → GAME_SESSION*	Obstacle_Type, Base_Speed, Min_Y_Position, Max_Y_Position, Spawn_Interval, Score_Value, Is_Active
SPIKES (OBSTACLE)	Spike_ID	Session_ID → GAME_SESSION; Obstacle_ID → OBSTACLE_CONFIG	CSOPaINwFnIG_Time, Y_Position, Move_Speed, Is_Destroyed, Destroyed_At
AUDIO_ASSET	Audio_ID	—	Audio_Name, Audio_Type, File_Path, Volume, Is_Loop, Created_At
PARALLAX_LAYER	Layer_ID	—	Layer_Name, Scroll_Speed, Sprite_Path, Order_In_Background, Is_Active

* **Important correction to the ER diagram:** the relationships shown for GAME_SESSION → DIFFICULTY_LEVEL and GAME_SESSION → OBSTACLE_CONFIG require corresponding foreign-key attributes if those relationships are to be implemented directly as 1:1 relationships. Add **Difficulty_ID** to GAME_SESSION and **Session_ID** to OBSTACLE_CONFIG. For GAME_SESSION → AUDIO_ASSET, if one session can use multiple audio assets, a junction table **SESSION_AUDIO**(**Session_ID**, **Audio_ID**) is the safer normalized design. If exactly one audio asset is allowed, Audio_ID can instead be placed in GAME_SESSION.

2D ARCADE GAME — DATABASE FINALIZATION

Final database design: The normalized entities are converted into implementation-ready relations. Primary keys uniquely identify records, while foreign keys enforce relationships shown in the ER model.

Final Relations / Tables

TABLE	FINAL PRIMARY KEY	FOREIGN KEY(S)	FINAL COLUMNS
PLAYER	Player_ID	—	Player_ID, Username, Email, Password, Hash, Created_At, Last_Login
GAME_SETTINGS	Setting_ID	Player_ID → PLAYER	Setting_ID, Player_ID, Sound_Volume, Music_Volume, Input_Method, Vibration, Graphics_Quality, Created_At, Updated_At
DIFFICULTY_LEVEL	Difficulty_ID	—	Difficulty_ID, Level_Number, Obstacle_Speed_Multiplier, Spawn_Interval_Multiplier, Score_Threshold, Description
GAME_SESSION	Session_ID	Player_ID → PLAYER; Difficulty_ID → DIFFICULTY_LEVEL	Session_ID, Player_ID, Difficulty_ID, Start_Time, End_Time, Final_Score, Max_Difficulty, Level_Reached_Is_Game_Over
LEADERBOARD	Entry_ID	Player_ID → PLAYER; Session_ID → GAME_SESSION	Entry_ID, Player_ID, Session_ID, Score, Level_Reached, Rank, Achieved_At
OBSTACLE_CONFIG	Obstacle_ID	—	Obstacle_ID, Obstacle_Type, Base_Speed, Min_Y_Position, Max_Y_Position, Spawn_Interval, Score_Value, Is_Active
SPIKES	Spike_ID	Session_ID → GAME_SESSION; Obstacle_ID → OBSTACLE_CONFIG	Spike_ID, Session_ID, Obstacle_ID, Spawn_Time, Y_Position, Move_Speed, Is_Destroyed, Destroyed_At
AUDIO_ASSET	Audio_ID	—	Audio_ID, Audio_Name, Audio_Type, File_Path, Volume, Is_Loop, Created_At
SESSION_AUDIO	Session_ID + Audio_ID	Session_ID → GAME_SESSION; Audio_ID → AUDIO_ASSET	Session_ID, Audio_ID
PARALLAX_LAYER	Layer_ID	—	Layer_ID, Layer_Name, Scroll_Speed, Sprite_Path, Order_In_Background, Is_Active

Final Relationship Structure

RELATIONSHIP	CARDINALITY	IMPLEMENTATION
PLAYER → GAME_SESSION	1 : N	GAME_SESSION.Player_ID
PLAYER → GAME_SETTINGS	1 : N	GAME_SETTINGS.Player_ID
PLAYER → LEADERBOARD	1 : N	LEADERBOARD.Player_ID
DIFFICULTY_LEVEL → GAME_SESSION	1 : N	GAME_SESSION.Difficulty_ID
GAME_SESSION → LEADERBOARD	1 : N	LEADERBOARD.Session_ID
GAME_SESSION → SPIKES	1 : N	SPIKES.Session_ID
OBSTACLE_CONFIG → SPIKES	1 : N	SPIKES.Obstacle_ID
GAME_SESSION ↔ AUDIO_ASSET	M : N	SESSION_AUDIO junction table

Finalization decisions: (1) Difficulty_ID is added to GAME_SESSION because a session progresses through one difficulty level. (2) Session_ID is added to SPIKES because each spike belongs to a game session. (3) SESSION_AUDIO is added because the ER relationship between GAME_SESSION and AUDIO_ASSET is many-to-many; this avoids repeating Audio_ID values and keeps the database normalized. (4) OBSTACLE_CONFIG is treated as reusable configuration data, while SPIKES stores the actual obstacle instances spawned during a session.

Result: The final design is suitable for implementation as a relational database and maintains 1NF, 2NF and 3NF under the stated assumptions.

Conclusion :

In conclusion, Floaty Astronaut is a simple yet engaging 2D space-themed obstacle-avoidance game developed using the Unity game engine. The game successfully combines easy-to-understand controls with challenging gameplay, requiring players to use quick reactions and precise movements to avoid spike-covered obstacles. The survival and score-based system provides motivation for players to continue playing and improve their performance. Overall, the project demonstrates how Unity can be used to create an accessible and enjoyable 2D arcade-style game while applying fundamental concepts of game development.